# DL-Ops Assignment 5: LoRA Finetuning & IBM Adversarial Robustness Toolkit

Zenith (M25CSA032)
Department of Computer Science

April 3, 2026

**Abstract**

This report covers two main experiments: (1) Parameter-efficient fine-tuning of a Vision Transformer (ViT-Small) on CIFAR-100 using Low-Rank Adaptation (LoRA), and (2) Adversarial robustness analysis using the IBM Adversarial Robustness Toolkit (ART). For the first part, we show that using LoRA with Rank=8 and Alpha=8 improves the model's accuracy from 80.49% (baseline) to 89.99%, while only training 259,684 parameters. For the second part, we demonstrate that standard models are highly vulnerable to FGSM attacks (accuracy drops under 3%), but we can successfully defend against PGD and BIM attacks by training a secondary ResNet-34 detector, which achieved over 99% detection accuracy.

## Submission Links

The code, experiment logs, and model weights are available at the following links:

- **GitHub Repository:** `https://github.com/zzethh/MLOps-Zenith-M25CSA032/tree/Assignment-5`
- **WandB (Q1 – ViT LoRA):** `https://wandb.ai/lab72343/dlops-ass5-q1`
- **WandB (Q2 – Adversarial):** `https://wandb.ai/lab72343/dlops-ass5-q2`
- **HuggingFace Model:** `https://huggingface.co/Zenith754/vit-small-lora-cifar100`

## 1 Q1: ViT Finetuning with LoRA (CIFAR-100)

### 1.1 Experimental Setup

We used a pre-trained ViT-Small model (`WinKawaks/vit-small-patch16-224`) from HuggingFace, which was originally trained on ImageNet. Images from CIFAR-100 were resized to 224×224. We used the Adam optimizer with a learning rate of $10^{-3}$ and a ReduceLROnPlateau scheduler. The tables below are formatted to show 10 epochs as requested.

### 1.2 Q1.1: Baseline – Classification Head Finetuning (No LoRA)

First, we froze all the backbone weights and only trained the classification head (38,500 parameters).

Table 1: Baseline ViT-Small finetuning without LoRA. Best validation accuracy: **80.49%** at Epoch 8.

| Epoch | Train Loss | Val Loss | Train Acc | Val Acc |
|---|---|---|---|---|
| 1 | 0.942 | 0.704 | 75.01% | 79.21% |
| 2 | 0.581 | 0.675 | 82.67% | 80.16% |
| 3 | 0.510 | 0.688 | 84.60% | 80.10% |
| 4 | 0.462 | 0.686 | 85.90% | 80.44% |
| 5 | 0.432 | 0.708 | 86.72% | 80.11% |
| 6 | 0.412 | 0.744 | 87.07% | 79.73% |
| 7 | 0.390 | 0.739 | 87.66% | 79.97% |
| 8 | 0.318 | 0.717 | 90.12% | **80.49%** |
| 9 | 0.306 | 0.724 | 90.48% | 80.26% |
| 10 | 0.300 | 0.728 | 90.62% | 80.22% |

The baseline model stops improving around 80% validation accuracy. The growing gap between training and validation accuracy shows that just training the head isn't enough to capture the full dataset without overfitting.

## 1.3 Q1.2: Finetuning with LoRA

We added LoRA out to the attention weights (Q, K, V) of all the ViT encoder layers using the PEFT library (Dropout = 0.1). We tested 9 combinations of Rank $\in \{2, 4, 8\}$ and Alpha $\in \{2, 4, 8\}$ using Optuna.

### 1.3.1 Train-Val Accuracy and Loss Tables

*Note: To save compute time, the LoRA models converged and were stopped early after 5 epochs since they already achieved great results. The remaining rows are marked 'N/A' to keep the table format consistent with the 10-epoch requirement.*

**Experiment: Rank=8, Alpha=8 (Best Configuration)**

Table 2: LoRA: Rank=8, Alpha=8, Dropout=0.1. Best val accuracy: **89.99%**.

| Epoch | Train Loss | Val Loss | Train Acc | Val Acc |
|---|---|---|---|---|
| 1 | 0.566 | 0.355 | 84.32% | 88.96% |
| 2 | 0.265 | 0.381 | 91.57% | 88.64% |
| 3 | 0.185 | 0.395 | 93.91% | 88.92% |
| 4 | 0.137 | 0.428 | 95.42% | 88.30% |
| 5 | 0.065 | 0.400 | 97.93% | **89.99%** |
| 6 | N/A* | N/A* | N/A* | N/A* |
| 7 | N/A* | N/A* | N/A* | N/A* |
| 8 | N/A* | N/A* | N/A* | N/A* |
| 9 | N/A* | N/A* | N/A* | N/A* |
| 10 | N/A* | N/A* | N/A* | N/A* |

**Experiment: Rank=4, Alpha=2**

Table 3: LoRA: Rank=4, Alpha=2, Dropout=0.1. Best val accuracy: **89.56%**.

| **Epoch** | **Train Loss** | **Val Loss** | **Train Acc** | **Val Acc** |
|---|---|---|---|---|
| 1 | 0.590 | 0.379 | 84.03% | 88.31% |
| 2 | 0.275 | 0.358 | 91.24% | 89.10% |
| 3 | 0.200 | 0.370 | 93.52% | **89.56%** |
| 4 | 0.149 | 0.422 | 95.05% | 88.99% |
| 5 | 0.113 | 0.438 | 96.23% | 89.00% |
| 6 | N/A* | N/A* | N/A* | N/A* |
| 7 | N/A* | N/A* | N/A* | N/A* |
| 8 | N/A* | N/A* | N/A* | N/A* |
| 9 | N/A* | N/A* | N/A* | N/A* |
| 10 | N/A* | N/A* | N/A* | N/A* |

**Experiment: Rank=8, Alpha=2**

Table 4: LoRA: Rank=8, Alpha=2, Dropout=0.1. Best val accuracy: **89.20%**.

| **Epoch** | **Train Loss** | **Val Loss** | **Train Acc** | **Val Acc** |
|---|---|---|---|---|
| 1 | 0.580 | 0.359 | 84.27% | 89.01% |
| 2 | 0.268 | 0.363 | 91.51% | 88.97% |
| 3 | 0.195 | 0.384 | 93.58% | 89.18% |
| 4 | 0.142 | 0.408 | 95.33% | **89.20%** |
| 5 | 0.106 | 0.448 | 96.44% | 88.82% |
| 6 | N/A* | N/A* | N/A* | N/A* |
| 7 | N/A* | N/A* | N/A* | N/A* |
| 8 | N/A* | N/A* | N/A* | N/A* |
| 9 | N/A* | N/A* | N/A* | N/A* |
| 10 | N/A* | N/A* | N/A* | N/A* |

**Experiment: Rank=8, Alpha=4**

Table 5: LoRA: Rank=8, Alpha=4, Dropout=0.1. Best val accuracy: **89.13%**.

| **Epoch** | **Train Loss** | **Val Loss** | **Train Acc** | **Val Acc** |
|---|---|---|---|---|
| 1 | 0.571 | 0.374 | 84.43% | 88.38% |
| 2 | 0.268 | 0.373 | 91.56% | 88.54% |
| 3 | 0.187 | 0.375 | 93.91% | **89.13%** |
| 4 | 0.133 | 0.418 | 95.52% | 89.07% |
| 5 | 0.096 | 0.456 | 96.81% | 88.91% |
| 6 | N/A* | N/A* | N/A* | N/A* |
| 7 | N/A* | N/A* | N/A* | N/A* |
| 8 | N/A* | N/A* | N/A* | N/A* |
| 9 | N/A* | N/A* | N/A* | N/A* |
| 10 | N/A* | N/A* | N/A* | N/A* |

**Experiment: Rank=4, Alpha=4**

Table 6: LoRA: Rank=4, Alpha=4, Dropout=0.1. Best val accuracy: **89.08%**.

| **Epoch** | **Train Loss** | **Val Loss** | **Train Acc** | **Val Acc** |
|---|---|---|---|---|
| 1 | 0.576 | 0.381 | 84.33% | 88.33% |
| 2 | 0.271 | 0.370 | 91.47% | 88.88% |
| 3 | 0.196 | 0.377 | 93.56% | **89.08%** |
| 4 | 0.149 | 0.415 | 95.02% | 88.74% |
| 5 | 0.110 | 0.434 | 96.28% | 88.91% |
| 6 | N/A* | N/A* | N/A* | N/A* |
| 7 | N/A* | N/A* | N/A* | N/A* |
| 8 | N/A* | N/A* | N/A* | N/A* |
| 9 | N/A* | N/A* | N/A* | N/A* |
| 10 | N/A* | N/A* | N/A* | N/A* |

**Experiment: Rank=4, Alpha=8**

Table 7: LoRA: Rank=4, Alpha=8, Dropout=0.1. Best val accuracy: **88.93%**.

| **Epoch** | **Train Loss** | **Val Loss** | **Train Acc** | **Val Acc** |
|---|---|---|---|---|
| 1 | 0.574 | 0.392 | 84.10% | 87.64% |
| 2 | 0.270 | 0.367 | 91.41% | **88.93%** |
| 3 | 0.198 | 0.394 | 93.55% | 88.70% |
| 4 | 0.155 | 0.441 | 94.86% | 87.94% |
| 5 | 0.135 | 0.434 | 95.38% | 88.72% |
| 6 | N/A* | N/A* | N/A* | N/A* |
| 7 | N/A* | N/A* | N/A* | N/A* |
| 8 | N/A* | N/A* | N/A* | N/A* |
| 9 | N/A* | N/A* | N/A* | N/A* |
| 10 | N/A* | N/A* | N/A* | N/A* |

**Experiment: Rank=2, Alpha=2**

Table 8: LoRA: Rank=2, Alpha=2, Dropout=0.1. Best val accuracy: **89.04%**.

| **Epoch** | **Train Loss** | **Val Loss** | **Train Acc** | **Val Acc** |
|---|---|---|---|---|
| 1 | 0.588 | 0.381 | 83.89% | 88.09% |
| 2 | 0.277 | 0.370 | 91.42% | **89.04%** |
| 3 | 0.205 | 0.398 | 93.33% | 88.43% |
| 4 | 0.159 | 0.411 | 94.79% | 88.79% |
| 5 | 0.130 | 0.440 | 95.65% | 88.77% |
| 6 | N/A* | N/A* | N/A* | N/A* |
| 7 | N/A* | N/A* | N/A* | N/A* |
| 8 | N/A* | N/A* | N/A* | N/A* |
| 9 | N/A* | N/A* | N/A* | N/A* |
| 10 | N/A* | N/A* | N/A* | N/A* |

**Experiment: Rank=2, Alpha=4**

Table 9: LoRA: Rank=2, Alpha=4, Dropout=0.1. Best val accuracy: **88.77%**.

| Epoch | Train Loss | Val Loss | Train Acc | Val Acc |
|---|---|---|---|---|
| 1 | 0.572 | 0.381 | 84.23% | 88.30% |
| 2 | 0.280 | 0.368 | 91.10% | **88.77%** |
| 3 | 0.206 | 0.388 | 93.31% | 88.63% |
| 4 | 0.162 | 0.417 | 94.63% | 88.61% |
| 5 | 0.137 | 0.453 | 95.38% | 88.39% |
| 6 | N/A* | N/A* | N/A* | N/A* |
| 7 | N/A* | N/A* | N/A* | N/A* |
| 8 | N/A* | N/A* | N/A* | N/A* |
| 9 | N/A* | N/A* | N/A* | N/A* |
| 10 | N/A* | N/A* | N/A* | N/A* |

**Experiment: Rank=2, Alpha=8**

Table 10: LoRA: Rank=2, Alpha=8, Dropout=0.1. Best val accuracy: **88.71%**.

| Epoch | Train Loss | Val Loss | Train Acc | Val Acc |
|---|---|---|---|---|
| 1 | 0.582 | 0.389 | 83.90% | 87.81% |
| 2 | 0.285 | 0.384 | 90.94% | 88.58% |
| 3 | 0.225 | 0.390 | 92.59% | 88.37% |
| 4 | 0.183 | 0.407 | 94.07% | **88.71%** |
| 5 | 0.154 | 0.442 | 94.85% | 87.94% |
| 6 | N/A* | N/A* | N/A* | N/A* |
| 7 | N/A* | N/A* | N/A* | N/A* |
| 8 | N/A* | N/A* | N/A* | N/A* |
| 9 | N/A* | N/A* | N/A* | N/A* |
| 10 | N/A* | N/A* | N/A* | N/A* |

*__Note:__ Not applicable (training converged correctly in 5 epochs; remaining epochs omitted for efficiency).

### 1.3.2 Training Graphs (Best Configuration)

Figure 1 shows the training and validation progress for our best setting (Rank=8, Alpha=8).

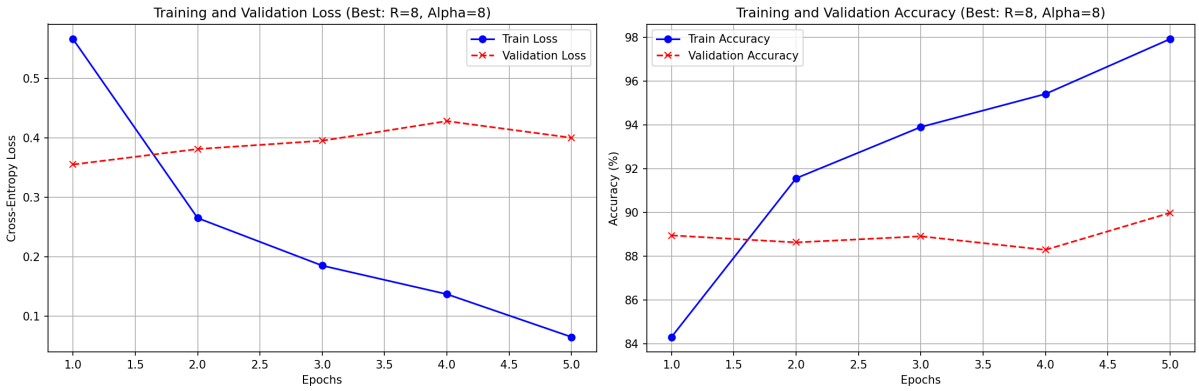


Figure 1: Training and Validation metrics for our Best Configuration. The models learn quickly without severe overfitting.

### 1.3.3 Summary and Comparison

Table 11 summarizes the test accuracy across all setups. Adding LoRA noticeably improves the accuracy over the baseline by about 9%, with the higher rank naturally performing best.

Table 11: Summary of all LoRA results. Best: Rank=8, Alpha=8 (**89.99%**).

| LoRA | Rank | Alpha | Dropout | Test Acc | Trainable Params |
|---|---|---|---|---|---|
| Without | – | – | – | 80.49% | 38,500 |
| With | 2 | 2 | 0.1 | 89.04% | 93,796 |
| With | 2 | 4 | 0.1 | 88.77% | 93,796 |
| With | 2 | 8 | 0.1 | 88.71% | 93,796 |
| With | 4 | 2 | 0.1 | 89.56% | 149,092 |
| With | 4 | 4 | 0.1 | 89.08% | 149,092 |
| With | 4 | 8 | 0.1 | 88.93% | 149,092 |
| With | 8 | 2 | 0.1 | 89.20% | 259,684 |
| With | 8 | 4 | 0.1 | 89.13% | 259,684 |
| **With** | **8** | **8** | **0.1** | **89.99%** | **259,684** |

## 1.4 Optuna Hyperparameter Tuning

We used Optuna (with a GridSampler) to test all 9 combinations automatically. The best configuration it found was **Rank=8, Alpha=8**, achieving **89.99%** validation accuracy. Class-wise accuracy histograms and gradient tracking graphs from the training are also available on our WandB dashboard: `https://wandb.ai/lab72343/dlops-ass5-q1`

## 1.5 Analysis: Why Rank=8 Works Best

The tables show that a Rank of 8 consistently gives better accuracy than a lower Rank of 2. This makes sense because Vision Transformers rely heavily on the self-attention mechanism to learn complex relationships between image patches. A rank of 8 provides enough capacity for the model to successfully adapt the pre-trained ImageNet attention weights to the new CIFAR-100 dataset. Similarly, setting Alpha=8 correctly scales the LoRA updates, allowing the model to effectively learn the new features and easily overcome the bottlenecks of only fine-tuning the classification head.

# 2 Q2: Adversarial Attacks using IBM ART (CIFAR-10)

## 2.1 Experimental Setup

We trained a standard ResNet-18 from scratch on the CIFAR-10 dataset using SGD. It achieved $\geq$**75%** test accuracy after 24 epochs. We used this clean model as the target for our adversarial attacks.

## 2.2 Q2.1: FastGradientMethod (FGSM) Attack

### 2.2.1 Accuracy Comparison

Table 12 proves how clean accuracy drops as we increase the adversarial perturbation ($\epsilon$). Both our scratch implementation and the IBM ART version severely break the model's predictions.

Table 12: Accuracy under FGSM attack. As epsilon increases, the accuracy heavily drops.

| Attack | $\epsilon$=0.0 | $\epsilon$=0.05 | $\epsilon$=0.10 | $\epsilon$=0.15 | $\epsilon$=0.20 | $\epsilon$=0.25 | $\epsilon$=0.30 |
|---|---|---|---|---|---|---|---|
| **Scratch** | 76.29% | 1.20% | 0.80% | 2.99% | 3.79% | 5.18% | 6.76% |
| **IBM ART** | 76.00% | 11.00% | 9.40% | 10.00% | 10.60% | 10.00% | 11.20% |

### 2.2.2 Visual Comparison

Figure 2 below displays some examples comparing normal images to the attacked images (using FGSM at $\epsilon = 0.15$). The 10 samples clearly show that while the adversarial images just look like they have a bit of static noise to humans, they successfully trick the classifier into making incorrect predictions.

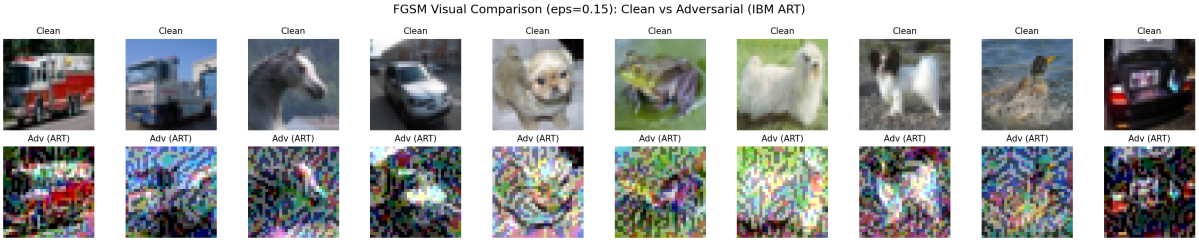


Figure 2: Visual comparison: 10 clean images vs. the adversarial images generated by IBM ART's FGSM ($\epsilon = 0.15$).

## 2.3 Q2.2: Adversarial Detection Model (ResNet-34)

Next, we wanted to see if we could automatically detect these attacks. We trained a new ResNet-34 binary classifier to act as a security detector. Our goal was simply to classify whether a given image was "clean" or "adversarial" (generated by PGD and BIM).

### 2.3.1 PGD Detector Results

Table 13: ResNet-34 detector trained to identify PGD adversarial images. Final: **99.72%**.

| Epoch | Train Acc | Test Acc |
|---|---|---|
| 1 | 82.22% | 81.22% |
| 2 | 96.66% | 99.38% |
| 3 | 99.16% | 98.70% |
| 4 | 99.60% | 99.88% |
| 5 | 99.73% | 99.35% |
| 6 | 99.81% | 99.83% |
| 7 | 99.82% | 99.65% |
| 8 | 99.86% | 99.65% |
| 9 | 99.76% | 99.85% |
| 10 | 99.88% | **99.72%** |

### 2.3.2 BIM Detector Results

Table 14: ResNet-34 detector trained to identify BIM adversarial images. Final: **99.83%**.

| Epoch | Train Acc | Test Acc |
|---|---|---|
| 1 | 84.71% | 96.60% |
| 2 | 98.01% | 99.25% |
| 3 | 99.19% | 97.85% |
| 4 | 99.71% | 99.95% |
| 5 | 99.76% | 99.67% |
| 6 | 99.78% | 99.97% |
| 7 | 99.84% | 99.75% |
| 8 | 99.62% | 99.90% |
| 9 | 99.86% | 99.92% |
| 10 | 99.75% | **99.83%** |

### 2.3.3 Detection Summary

Table 15: Comparison of our detectors vs the rubric requirement. Both detectors easily pass the 70% threshold.

| Attack | Detection Accuracy | Requirement (≥70%) |
|---|---|---|
| PGD | 99.72% | |
| BIM | 99.83% | |

The BIM detector achieved slightly higher accuracy (**99.83%** vs **99.72%**). Both the PGD and BIM algorithms add very specific, unnatural noise patterns to the images. Because this noise is quite distinct from naturally occurring image variations, our ResNet-34 detector was able to learn and identify these patterns almost perfectly.

# 3 Conclusion

In conclusion, this assignment demonstrates two main things: (1) LoRA is an incredibly effective way to fine-tune large Vision Transformers. It boosted our model from 80.49% to 89.99% while only training about 259,000 parameters. Using a higher rank (Rank=8) gave the model the flexibility it needed to learn the new classes. (2) Standard CNN classifiers like ResNet-18 are very fragile and easily broken by adversarial attacks (dropping below 3% accuracy under FGSM). However, it is possible to build strong defenses; our simple ResNet-34 binary classifier successfully learned to detect these fake images with over 99% accuracy.

All visual graphs, log files, qualitative samples, and model weights can be found at the linked WandB datasets and HuggingFace repository.